

TECNOLÓGICO NACIONAL DE MÉXICO

Gestión de Proyectos

IMPLEMENTACIÓN DEL BACKEND (API DE INGESTA)

Equipo 3A

Bolaños García Omar Gadiel

Diego Emilio Ortiz Vilchez

Perez Manriques Jorge Luis

Chavez Vega Emilio Sebastian

ING. Sistemas Computacionales | Celaya

Proyecto: Monitor de Nivel de Agua en Tinaco

KR 3.5: Implementación del Backend (API de Ingesta)

Entregable: Documentación del Código Funcional del Servidor

Sprint 3 | Fecha: 13 may 2026 | **Estado:** In Review

1. Objetivo

Desarrollar e implementar la lógica del lado del servidor necesaria para procesar las peticiones HTTP provenientes del hardware. Esto incluye la creación de controladores (controllers) que gestionen la recepción de datos, la aplicación de la lógica de negocio (validación y limpieza) y la persistencia final en Firebase.

2. Arquitectura del Servidor

Se utiliza un patrón de diseño desacoplado para asegurar que la API sea mantenible y escalable:

- Rutas (Routes):** Definen los endpoints REST descritos en el KR 3.3.
- Middleware:** Capa de seguridad y validación de esquemas (KR 3.4).
- Controladores (Controllers):** Orquestan la recepción del Request y la respuesta al cliente.
- Servicios (Services):** Ejecutan la lógica de negocio y la comunicación con la SDK de Firebase.

3. Implementación del Controlador Principal

A continuación se presenta la implementación lógica (en Node.js/TypeScript) para el procesamiento de las lecturas de nivel:

```

import { Request, Response } from 'express';
import { db } from '../config/firebase';

export const updateWaterLevel = async (req: Request, res: Response) => {
  try {
    const data = req.body;

    // 1. Lógica de Negocio: Truncar decimales por seguridad
    const sanitizedData = {
      ...data,
      p: parseFloat(data.p.toFixed(1)),
      n: parseFloat(data.n.toFixed(1)),
      updatedAt: new Date().toISOString()
    };

    // 2. Persistencia en el nodo de lectura actual
    await db.ref('tinaco/lectura').set(sanitizedData);

    // 3. Respuesta exitosa según contrato KR 3.3
    return res.status(200).json({ status: 'success', data: sanitizedData });

  } catch (error) {
    return res.status(500).json({ error: 'Internal Server Error' });
  }
};

```

4. Lógica de Disparo (Triggers) y Historial

Para evitar saturar la base de datos con 172,800 registros diarios en el historial, el backend implementa una ventana de tiempo o "throttle":

- **Actualización en Tiempo Real:** El nodo /lectura se actualiza cada 500ms (PUT).
- **Persistencia Histórica:** Una Cloud Function monitorea el nodo y guarda una muestra en /historial solo cada 5 minutos, o cuando se detecta un cambio abrupto (>10%) en el nivel.

5. Pruebas de Integración (Postman/cURL)

Se validó el funcionamiento del servidor mediante la siguiente prueba de carga básica:

Prueba	Entrada	Resultado Esperado	Estado
Ingesta Normal	JSON Válido + Token	200 OK - Datos en Firebase	Pasó
Datos Malformados	JSON con claves extras	400 Bad Request (Middleware)	Pasó
Sin Autenticación	Request sin Header	401 Unauthorized	Pasó

6. Conclusiones

El backend ahora es funcional y cumple estrictamente con los contratos de API y seguridad definidos previamente. La separación de responsabilidades entre el controlador y el servicio de base de datos asegura que cambios futuros en la infraestructura de nube no afecten la lógica de ingesta del hardware.